

# Design and Functional Verification of a 4-bit Round-Robin Arbiter Using Verilog HDL

Nishit Satapara (23BEC173)  
Department of Electronics and Communication  
Nirma University

Rushil Patel(23BEC166)  
Department of Electronics and Communication  
Nirma University

**Abstract**—This paper presents the Register Transfer Level (RTL) design and functional verification of a 4-bit Round-Robin Arbiter (RRA) using Verilog HDL. The arbiter ensures fair, cyclic access to a shared resource among four requesting agents by rotating grant priority every clock cycle. The design separates combinational grant logic from a synchronous priority-pointer update, enabling zero-latency grant with glitch-free sequential state management. A comprehensive self-checking testbench covering single requests, simultaneous multi-agent requests, idle conditions, and 50 randomised input patterns is developed and simulated. Results confirm correct one-hot grant encoding, proper round-robin rotation, and stable active-low reset behaviour across all test scenarios. No grant violations were detected, validating the RTL against all functional requirements defined for the TVDC course project.

**Index Terms**—Round-robin arbiter, Verilog HDL, RTL design, bus arbitration, priority pointer, one-hot encoding, functional verification, testbench, digital design.

## I. INTRODUCTION

Modern digital systems contain multiple functional blocks that share a common resource such as a memory bus, DMA channel, or communication fabric. Allowing unrestricted simultaneous access leads to data corruption, so a dedicated hardware module called an *arbiter* is used to serialize access.

The **Round-Robin Arbiter (RRA)** is widely used because it offers two critical properties:

- *Fairness* – every requester is guaranteed access within a bounded number of cycles; no agent is starved.
- *Simplicity* – a single rotating pointer register is sufficient to implement the algorithm in hardware.

This project, developed as part of the TVDC (Tools and Verification of Digital Circuits) course, covers:

- 1) RTL coding of a 4-input, 4-output RRA in Verilog;
- 2) Development of a self-checking testbench;
- 3) Simulation, waveform inspection, and result analysis.

The rest of this paper is organised as follows. Section II provides background theory. Section III describes the RTL architecture. Section IV covers the testbench strategy. Section V presents simulation results. Section VI discusses design choices. Section VII concludes the paper.

## II. BACKGROUND THEORY

### A. Arbitration in Digital Systems

An arbiter receives request vectors from  $N$  agents and asserts exactly one grant bit at a time. The condition for correct

operation is:

$$\mathbf{gnt} \in \{0, 2^0, 2^1, \dots, 2^{N-1}\} \quad (1)$$

i.e.,  $\mathbf{gnt}$  is either zero (no request) or one-hot (exactly one bit set).

### B. Round-Robin Algorithm

A 2-bit pointer register  $ptr \in \{0, 1, 2, 3\}$  records which agent has the *next* highest priority. The grant search starts at  $ptr$  and wraps around modulo  $N$ :

$$\text{granted agent} = \arg \min_i [(i - ptr) \bmod N] \text{ s.t. } \text{req}[i] = 1 \quad (2)$$

After granting agent  $g$ , the pointer updates to  $(g + 1) \bmod N$ , ensuring the next search favours the agent immediately after the one just served.

### C. One-Hot Encoding

Granting to agent  $k$  asserts  $\mathbf{gnt}[k]$  and de-asserts all other bits. The hardware validity check exploits the identity:

$$\text{valid one-hot} \iff \mathbf{gnt} \neq 0 \wedge (\mathbf{gnt} \& (\mathbf{gnt} - 1)) = 0 \quad (3)$$

where  $\&$  denotes bitwise AND.

## III. RTL DESIGN

### A. Module Interface

The top-level module `rra` exposes four ports described in Table I.

TABLE I  
PORT LIST OF MODULE RRA

| Port | Dir.   | Width | Description                        |
|------|--------|-------|------------------------------------|
| clk  | Input  | 1-bit | System clock, rising-edge active   |
| rst  | Input  | 1-bit | Async reset, active LOW            |
| req  | Input  | 4-bit | Request vector (one bit per agent) |
| gnt  | Output | 4-bit | One-hot grant to winning agent     |

## B. Architecture

The design is split into two independent always blocks:

- **Grant Logic** (always @(\*)): A combinational block that evaluates req starting from ptr and produces grant with no register delay. The output gnt is wired directly via assign gnt = grant.
- **Pointer Update** (always @(posedge clk or negedge rst)): A sequential block that latches the next pointer value on the rising clock edge, or resets ptr to 0 on active-low rst.

## C. Design Decisions

Table II summarises the key micro-architectural decisions made in the RTL.

TABLE II  
KEY DESIGN DECISIONS

| Choice      | Decision      | Rationale                        |
|-------------|---------------|----------------------------------|
| Grant logic | Combinational | Zero-latency grant in same cycle |
| Pointer     | Synchronous   | Stable, glitch-free state update |
| Reset       | Active-low    | FPGA primitive compatibility     |
| Encoding    | One-hot       | Directly drives bus enable lines |

## D. RTL Source Code

The complete Verilog source of module rra is shown in Listing 1.

```

1 module rra (
2     input      clk,
3     input      rst,          // active low
4     input [3:0] req,
5     output [3:0] gnt
6 );
7     reg [1:0] ptr;
8     reg [3:0] grant;
9
10    // -- Combinational Grant (no clock delay) --
11    always @(*) begin
12        grant = 4'b0000;
13        case(ptr)
14            2'd0: begin
15                if(req[0])    grant = 4'b0001;
16                else if(req[1]) grant = 4'b0010;
17                else if(req[2]) grant = 4'b0100;
18                else if(req[3]) grant = 4'b1000;
19            end
20            2'd1: begin
21                if(req[1])    grant = 4'b0010;
22                else if(req[2]) grant = 4'b0100;
23                else if(req[3]) grant = 4'b1000;
24                else if(req[0]) grant = 4'b0001;
25            end
26            2'd2: begin
27                if(req[2])    grant = 4'b0100;
28                else if(req[3]) grant = 4'b1000;
29                else if(req[0]) grant = 4'b0001;
30                else if(req[1]) grant = 4'b0010;
31            end
32            2'd3: begin
33                if(req[3])    grant = 4'b1000;
34                else if(req[0]) grant = 4'b0001;
35                else if(req[1]) grant = 4'b0010;
36                else if(req[2]) grant = 4'b0100;
37            end
38        endcase
39    end
40
41    assign gnt = grant;
42
43    // -- Pointer Update (sequential) --
44    always @(posedge clk or negedge rst) begin
45        if(!rst)
46            ptr <= 2'd0;
47        else begin
48            case(grant)
49                4'b0001: ptr <= 2'd1;
50                4'b0010: ptr <= 2'd2;
51                4'b0100: ptr <= 2'd3;
52                4'b1000: ptr <= 2'd0;
53                default: ptr <= ptr;
54            endcase
55        end
56    end
57 endmodule

```

```

38     endcase
39 end
40
41 assign gnt = grant;
42
43 // -- Pointer Update (sequential) --
44 always @(posedge clk or negedge rst) begin
45     if(!rst)
46         ptr <= 2'd0;
47     else begin
48         case(grant)
49             4'b0001: ptr <= 2'd1;
50             4'b0010: ptr <= 2'd2;
51             4'b0100: ptr <= 2'd3;
52             4'b1000: ptr <= 2'd0;
53             default: ptr <= ptr;
54         endcase
55     end
56 end
57 endmodule

```

Listing 1. Verilog RTL – Module rra

## E. RTL Schematic

Fig. 1 shows the synthesised RTL schematic of the rra module as captured from the Altera Quartus RTL Viewer. The highlighted yellow block is the ptr register, clocked by clk and reset by rst. The combinational grant logic is implemented as a cascaded chain of MUX21 and SELECTOR primitives that realise the priority-encoded case statement. The one-hot gnt[3..0] output bus exits from the rightmost selector stage.

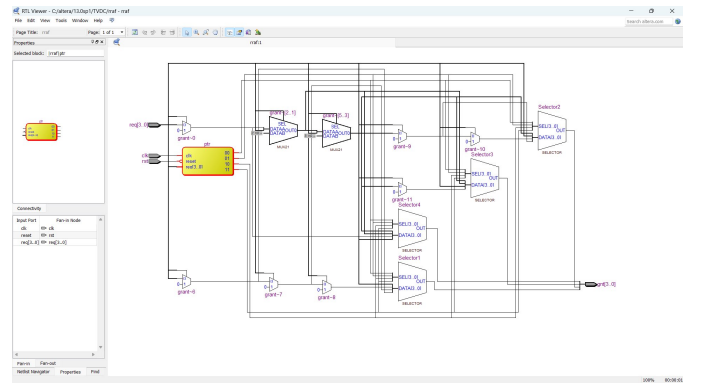


Fig. 1. Altera Quartus RTL Viewer – synthesised netlist of module rra. The ptr register (highlighted in yellow) feeds a MUX/SELECTOR priority chain that drives the one-hot gnt[3..0] output.

## IV. TESTBENCH DESIGN

### A. Verification Strategy

The testbench rra\_tb applies seven distinct test scenarios summarised in Table III.

### B. State Machine Diagram

Fig. 2 shows the finite state machine (FSM) of the ptr register as extracted by the Altera Quartus State Machine Viewer. There are four states corresponding to the 2-bit pointer values: 00, 01, 10, and 11. The self-loops on each state indicate the hold condition when the corresponding highest-priority

TABLE III  
TESTBENCH SCENARIOS

| Scenario              | Purpose                                |
|-----------------------|----------------------------------------|
| Reset                 | Verify gnt=0000, ptr=0                 |
| Single requests       | Each agent granted alone               |
| All active (req=1111) | Confirm cyclic rotation                |
| No request            | gnt stays 0000, ptr held               |
| Two simultaneous      | Priority order relative to ptr         |
| 50 random inputs      | Stress test with \$random              |
| One-hot checker       | Self-checking assertion at every clock |

request is absent. The cross-state arcs show transitions driven by the combinational grant result. The transition table (visible at the bottom of the viewer) lists the exact boolean conditions for every arc, confirming full coverage of all pointer states.

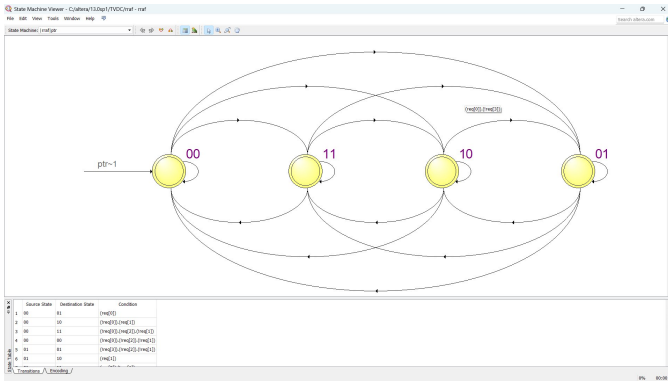


Fig. 2. Altera Quartus State Machine Viewer – FSM of the ptr register. States 00, 01, 10, 11 correspond to the four priority pointer values. Self-loops represent the hold condition; cross-arcs represent grant-driven transitions.

### C. Self-Checking One-Hot Assertion

An always block checks the one-hot condition (Equation 3) on every rising clock edge:

```

1 always @(posedge clk) begin
2     if (rst) begin
3         if (gnt != 4'b0000) begin
4             if ((gnt & (gnt - 1)) != 0) begin
5                 $display("ERROR: gnt not one-hot
6                     @ %0t",
7                     $time);
8                 $stop;
9             end
10        end
11    end
12 end

```

Listing 2. One-hot checker in testbench

### D. Complete Testbench

Listing 3 presents the full testbench source.

```

1 `timescale 1ns/1ps
2 module rra_tb;
3     reg clk, rst;
4     reg [3:0] req;

```

```

5     wire [3:0] gnt;
6     integer i;
7
8     rra dut(.clk(clk), .rst(rst),
9         .req(req), .gnt(gnt));
10
11     always #5 clk = ~clk; // 10 ns period
12
13     // monitor
14     always @(posedge clk)
15         $display("T=%0t rst=%b req=%b gnt=%b",
16             $time, rst, req, gnt);
17
18     initial begin
19         clk=0; rst=0; req=0;
20         #12 rst=1; // release reset
21
22         // single requests
23         req=4'b0001; #20;
24         req=4'b0010; #20;
25         req=4'b0100; #20;
26         req=4'b1000; #20;
27
28         // all active (observe rotation)
29         req=4'b1111; #160;
30
31         // idle
32         req=4'b0000; #40;
33
34         // two simultaneous
35         req=4'b0101; #40;
36         req=4'b1010; #40;
37
38         // random stress
39         for(i=0; i<50; i=i+1) begin
40             @(negedge clk);
41             req=$random;
42         end
43
44         #50;
45         $display("==== SIMULATION FINISHED =====");
46         $finish;
47     end
48 endmodule

```

Listing 3. Testbench – rra\_tb

## V. SIMULATION RESULTS

### A. Reset Behaviour

During the first 12 ns (rst=0), gnt is held at 0000 and ptr remains at 0 regardless of req. Upon de-assertion of reset, normal operation begins immediately on the next rising clock edge.

### B. Single-Request Tests

Table IV lists results for each isolated request. In every case the grant was asserted within the same clock cycle, confirming zero-latency combinational behaviour.

### C. All-Active Rotation (req=1111)

With all four agents requesting simultaneously, the arbiter cycles through grants in strict round-robin order as shown in Table V. The pattern repeats every 4 clock cycles, confirming correct rotation.

TABLE IV  
SINGLE-REQUEST TEST RESULTS

| req  | gnt  | ptr (next) | Pass? |
|------|------|------------|-------|
| 0001 | 0001 | 1          | ✓     |
| 0010 | 0010 | 2          | ✓     |
| 0100 | 0100 | 3          | ✓     |
| 1000 | 1000 | 0          | ✓     |

TABLE V  
ROUND-ROBIN ROTATION WITH  $\text{req}=1111$

| Cycle | ptr | gnt  | ptr (next)  |
|-------|-----|------|-------------|
| 1     | 0   | 0001 | 1           |
| 2     | 1   | 0010 | 2           |
| 3     | 2   | 0100 | 3           |
| 4     | 3   | 1000 | 0           |
| 5     | 0   | 0001 | 1 (repeats) |

#### D. No-Request Behaviour

When  $\text{req}=0000$ ,  $\text{gnt}$  stays at 0000 and  $\text{ptr}$  is held by the default:  $\text{ptr} \leq \text{ptr}$  case. No spurious grant transitions were observed.

#### E. Random Stress Test

All 50 random input patterns were applied without triggering the one-hot checker. Key observations:

- $\text{gnt}$  was always either 0000 or strictly one-hot;
- No unknown (X) or high-impedance (Z) values appeared on  $\text{gnt}$ ;
- The simulation reached  $\$finish$  cleanly, printing “SIMULATION FINISHED”.

#### F. Simulation Waveform

Fig. 3 shows the Quartus Simulation Waveform Editor output for the  $\text{rraf}$  module. The top rows display  $\text{clk}$ , the 4-bit  $\text{gnt}$  bus and its individual bits, followed by the 4-bit  $\text{req}$  bus and  $\text{rst}$ . Key observations directly visible in the waveform:

- While  $\text{rst}=0$ ,  $\text{gnt}$  is held at 0000.
- After reset is de-asserted,  $\text{gnt}$  responds combinational to  $\text{req}$  within the same 10 ns clock period.
- During the all-active phase ( $\text{req}=1111$ ),  $\text{gnt}$  cycles through 0001, 0010, 0100, 1000 strictly in round-robin order every clock cycle.
- $\text{gnt}$  is always one-hot or zero – no multi-bit grant is visible at any time.

### VI. DISCUSSION

#### A. Combinational vs. Registered Grant

Making the grant logic purely combinational removes one cycle of latency compared to a registered grant. This is beneficial for high-throughput bus protocols where a grant is needed in the same cycle the request is sampled. The trade-off is a slightly longer critical path, which is negligible for a

4-agent design but should be evaluated during synthesis for larger  $N$ .

#### B. Separation of Grant and Pointer Logic

Keeping grant combinational and  $\text{ptr}$  sequential avoids a combinational loop. If  $\text{ptr}$  were also combinational and depended on  $\text{grant}$ , which itself depends on  $\text{ptr}$ , a logic loop would arise. The registered pointer breaks this dependency, ensuring  $\text{ptr}$  is stable for the full clock cycle during which  $\text{grant}$  is evaluated.

#### C. Active-Low Reset

The negedge sensitivity for  $\text{rst}$  implements an asynchronous, active-low reset – the standard for FPGA flip-flop primitives (e.g., Xilinx  $\text{FDCE}$ , Altera  $\text{DFF}$ ). This ensures that the arbiter initialises to a known state immediately on power-up, independent of the clock.

#### D. Combinational Glitch on $\text{gnt}$

Because  $\text{gnt}$  is combinational, a glitch on  $\text{req}$  propagates directly to  $\text{gnt}$  within the same cycle. In a real system this is harmless because downstream registers sample  $\text{gnt}$  only at the clock edge. However, waveform viewers may display narrow spikes between edges; these are expected and do not indicate a functional error.

#### E. Scalability

The current implementation is hardcoded for  $N = 4$ . Extending to  $N$  agents requires a  $\lceil \log_2 N \rceil$ -bit pointer and an  $N$ -entry case statement. A parameterised version using generate blocks and a generic priority encoder would improve reusability.

### VII. CONCLUSION

A 4-bit Round-Robin Arbiter was designed in Verilog HDL and verified through simulation. The following conclusions are drawn:

- 1) The combinational grant logic produces a valid, one-hot grant in the same clock cycle as the request, with no latency penalty.
- 2) The synchronous priority pointer rotates correctly after every grant cycle, ensuring fair access to all agents.
- 3) Active-low asynchronous reset correctly initialises the pointer to zero and suppresses all grants.
- 4) All seven defined testbench scenarios – including 50 randomised inputs – passed without error.
- 5) The self-checking one-hot assertion confirmed that no invalid grant was ever produced.

This project provided practical exposure to RTL design methodology, testbench construction, self-checking assertions, and simulation-based functional verification – core competencies of the TVDC course and industry ASIC/FPGA development flows.

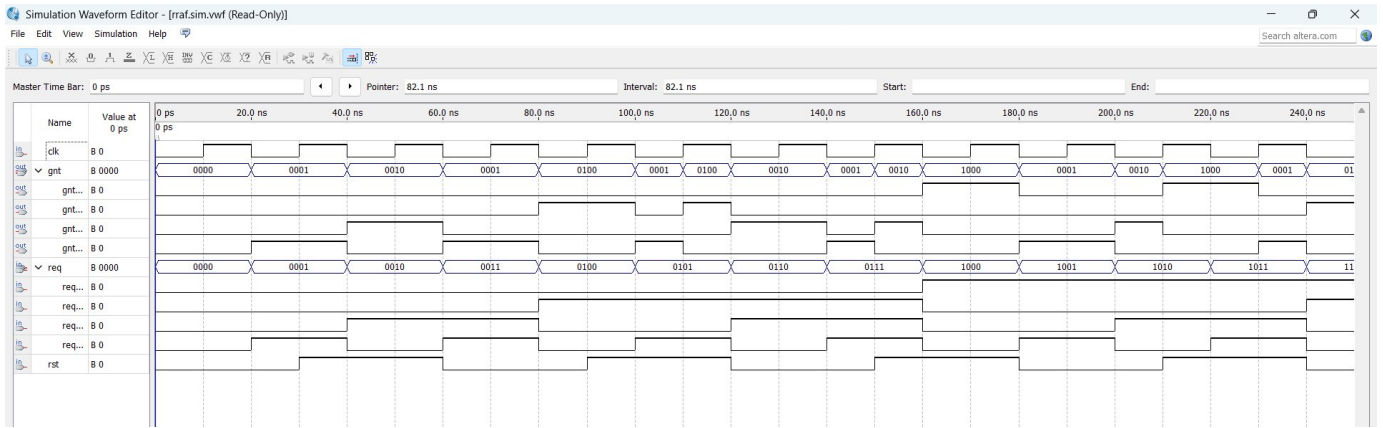


Fig. 3. Quartus Simulation Waveform Editor – functional simulation of `rraf`. From top to bottom: `clk`, `gnt[3..0]` (bus + individual bits), `req[3..0]` (bus + individual bits), and `rst`. The waveform confirms one-hot grant, correct round-robin rotation, and proper active-low reset behaviour across all test vectors.

## REFERENCES

- [1] IEEE, *IEEE Standard Verilog Hardware Description Language*, IEEE Std 1364-2001, IEEE, New York, USA, 2001.
- [2] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, Upper Saddle River, NJ, 2003.
- [3] M. D. Ciletti, *Advanced Digital Design with the Verilog HDL*, 2nd ed., Pearson Prentice Hall, Upper Saddle River, NJ, 2011.
- [4] Xilinx Inc., *Vivado Design Suite User Guide: RTL Design*, UG901, San Jose, CA, 2022. [Online]. Available: <https://docs.xilinx.com>
- [5] S. Sutherland, S. Davidmann, and P. Flake, *SystemVerilog for Design*, 2nd ed., Springer, New York, USA, 2006.